



Sprint 2: Data Preparation

Limpieza, Features y Pipelines

Sprint Backlog, Roles y Definition of Done

Pedro Shiguihara, pshiguihara@pucp.edu.pe

10 de abril de 2026

Agenda

1. Contexto: del entendimiento a la preparación de datos
2. Metodología: CRISP-DM y Scrum aplicados al Sprint 2
3. Product Backlog: ítems del Sprint 2
4. Sprint 2: Alcance y Sprint Planning
5. Roles del equipo (5 personas)
6. Detalle de tareas por rol
7. Entregables del Sprint 2
8. Criterios de aceptación y Definition of Done

Contexto del Sprint 2

Del Sprint 1 (entender) al Sprint 2 (preparar)

🎯 Objetivo del Sprint 2

Transformar el dataset **crudo** explorado en el Sprint 1 en un dataset **listo para modelar**: limpio, con features informativas, clases balanceadas y un **pipeline reproducible** de `scikit-learn` que pueda reutilizarse en los siguientes sprints.

➡ Insumos del Sprint 1

- `01_business.ipynb` (objetivo y KPIs)
- `02_data_loading.ipynb` (carga del dataset)
- `03_eda.ipynb` (calidad y hallazgos)
- `04_prototype.ipynb` (exploración interactiva)
- Hallazgos del EDA: nulos, outliers, desbalance

🔧 Herramientas Clave

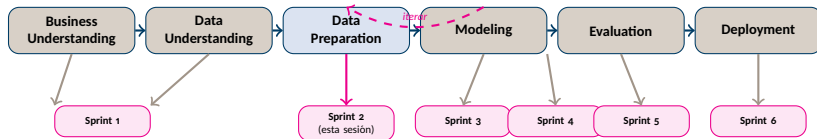
- Python 3.10+
- `scikit-learn` (Pipelines, ColumnTransformer)
- `imbalanced-learn` (SMOTE)
- `feature-engine`
- DVC (versionar datasets procesados)
- Git + GitHub Projects

⚠ Clave

No se entrena ningún modelo aún. El producto del Sprint 2 es un dataset transformado y un pipeline reproducible, no un clasificador.

Metodología: CRISP-DM dentro de Scrum

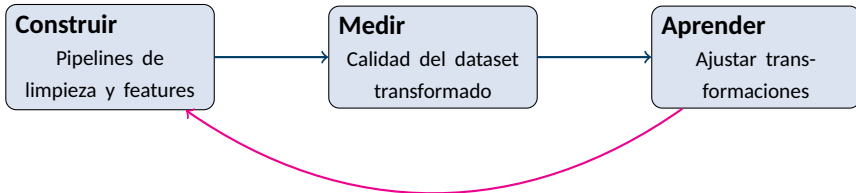
Mapeo de fases CRISP-DM a 6 Sprints — foco en Sprint 2



- **Sprint 1:** Business & Data Understanding.
- **Sprint 2:** Data Preparation (*esta sesión*).
- **Sprint 3:** Modeling (baseline).
- **Sprint 4:** Modeling avanzado + tuning.
- **Sprint 5:** Evaluation + Business Value.
- **Sprint 6:** Deployment en AWS (MVP).

Ciclo Lean Startup aplicado al Sprint 2

Construir → Medir → Aprender



💡 Filosofía del Sprint 2

Aún no se construye ningún modelo. El MVP de este sprint es un **pipeline reproducible** (`sklearn.Pipeline`) que toma los datos crudos y devuelve un dataset listo para modelar, junto con notebooks que documentan cada decisión de transformación.

Product Backlog Completo

Historias de usuario para todo el proyecto (6 sprints)

ID	Historia de Usuario	Fase CRISP-DM	Sprint
PB-01	Definir el problema de negocio y objetivos	Business Underst.	1
PB-02	Identificar criterios de éxito del modelo	Business Underst.	1
PB-03	Cargar y explorar el dataset inicial	Data Understanding	1
PB-04	Analizar calidad de datos (nulos, tipos, duplicados)	Data Understanding	1
PB-05	Generar estadísticas descriptivas y visualizaciones	Data Understanding	1
PB-06	Construir prototipo interactivo de exploración	Data Understanding	1
PB-07	Limpiar y transformar variables	Data Preparation	2
PB-08	Realizar feature engineering	Data Preparation	2
PB-09	Balancear clases si es necesario	Data Preparation	2
PB-10	Entrenar modelos baseline de clasificación	Modeling	3
PB-11	Evaluar modelos baseline con métricas	Modeling	3
PB-12	Comparar modelos y seleccionar candidatos	Modeling	3
PB-13	Optimizar hiperparámetros del mejor modelo	Modeling	4
PB-14	Aplicar técnicas avanzadas (ensambles, stacking)	Modeling	4
PB-15	Validar modelo final (cross-validation, test set)	Modeling	4
PB-16	Evaluar Business Value del modelo seleccionado	Evaluation	5
PB-17	Construir dashboard interactivo de resultados	Evaluation	5
PB-18	Documentar hallazgos y recomendaciones de negocio	Evaluation	5
PB-19	Empaquetar modelo como API (Flask/FastAPI)	Deployment	6
PB-20	Desplegar prototipo MVP en AWS (EC2/Lambda)	Deployment	6
PB-21	Integrar modelo con dashboard desplegado	Deployment	6

→ Las filas resaltadas en azul corresponden al **Sprint 2** (esta iteración): PB-07, PB-08 y PB-09.

Sprint 2: Sprint Planning

Alcance: Data Preparation completa y reproducible

Meta del Sprint

Producir un **dataset listo para modelar** y un **pipeline de scikit-learn** que aplique todas las transformaciones de forma reproducible, validado contra los hallazgos del EDA del Sprint 1.

Duración

- **1 semana** de trabajo colaborativo
- Daily stand-ups (async vía GitHub Issues)
- Sprint Review al final de la semana

Sprint Backlog (3 ítems grandes)

- PB-07** Limpieza y transformación de variables (nulos, outliers, tipos, encoding, escalado).
- PB-08** Feature engineering (nuevas variables, interacciones, binning, fechas).
- PB-09** Balanceo de clases (SMOTE, undersampling u otros, si aplica).

Gestión

- Mismo repositorio GitHub del Sprint 1
- Cada sub-tarea = 1 Issue en el board Kanban
- Ramas por feature: `feat/PB-XX`

Roles del Equipo: 5 Personas

Roles adaptados al Sprint 2 (Data Preparation)



Project Manager
Coordina, gestiona el Kanban y revisa entregables



Data Cleaner
Limpia nulos, outliers, duplicados y tipos (PB-07)



Feature Engineer
Crea nuevas variables y transformaciones (PB-08)



Class Balancer
Aplica SMOTE u otras técnicas de balanceo (PB-09)



Pipeline Builder
Integra todo en un `sklearn.Pipeline` reproducible

Nota importante

Los roles **rotan** respecto al Sprint 1: cada miembro del equipo asume un rol distinto para ganar experiencia en otra fase de CRISP-DM. Igualmente, todos colaboran y revisan el trabajo de los demás vía **Pull Requests**.

Rol 1: Project Manager

 Coordinación, planificación y seguimiento

Tareas del Sprint 2

1. Crear los Issues del Sprint 2 (PB-O7, PB-O8, PB-O9) y sub-issues por tarea.
2. Reorganizar el board Kanban (cerrar Sprint 1, abrir Sprint 2).
3. Asignar responsables y revisores cruzados a cada Issue.
4. Verificar que todos los PRs del Sprint 2 sean revisados antes del merge.
5. Coordinar la integración entre Data Cleaner, Feature Engineer y Pipeline Builder.
6. Preparar el Sprint Review: demo del pipeline + dataset transformado.

Estructura del Repo (Sprint 2)

```
proyecto/  
+-- data/  
|   +-- raw/  
|   +-- interim/  
|   +-- processed/  
+-- notebooks/  
|   +-- 05_data_cleaning.ipynb  
|   +-- 06_feature_eng.ipynb  
|   +-- 07_class_balance.ipynb  
|   +-- 08_pipeline.ipynb  
+-- src/  
|   +-- preprocessing.py  
+-- environment.yml
```

Rol 2: Data Cleaner

 Limpieza y transformación (PB-07)

Tareas del Sprint 2

1. Tratar **valores nulos** según su mecanismo (MCAR, MAR, MNAR):
 - Imputación por media/mediana/moda.
 - KNNImputer, IterativeImputer.
2. Detectar y tratar **outliers** (IQR, z-score, winsorizing).
3. Eliminar **duplicados** y registros inconsistentes.
4. Corregir **tipos de datos** (fechas, categóricos, numéricos).
5. Aplicar **encoding** a categóricas: OneHot, Ordinal, Target.
6. Aplicar **escalado**: StandardScaler, MinMaxScaler, RobustScaler.

Notebook

05_data_cleaning.ipynb

Debe contener:

- Resumen de hallazgos del EDA
- Decisión por cada problema (con justificación)
- Código aplicado a un **copy** del dataset
- Dataset intermedio guardado en data/interim/
- Verificación antes/después (shape, nulls, dtypes)



Tip

Nunca elimines variables sin justificar. Documenta el porqué en Markdown.

Rol 3: Feature Engineer

 Feature engineering (PB-o8)

Tareas del Sprint 2

1. Crear **features derivadas** a partir de variables existentes:
 - Ratios, diferencias, agregaciones.
 - Extraer componentes de fechas (año, mes, día, dow).
2. **Binning** de variables numéricas en categóricas.
3. Crear **interacciones** entre variables relevantes.
4. Aplicar **transformaciones**: log, Box-Cox, Yeo-Johnson para reducir asimetría.
5. **Selección** preliminar (eliminar redundancia con correlación, varianza nula).
6. Documentar cada feature nueva: nombre, cómo se calcula y por qué podría ser útil.

Ejemplo

```
import pandas as pd
import numpy as np

# Ratio
df['precio_x_noche'] = (
    df['precio_total'] /
    df['noches'])

# Componentes de fecha
df['mes'] = df['fecha'].dt.month
df['dow'] = df['fecha'].dt.dayofweek

# Binning
df['edad_grupo'] = pd.cut(
    df['edad'],
    bins=[0, 25, 45, 65, 100],
    labels=['joven', 'adulto',
            'maduro', 'senior'])

# Log para reducir skew
df['ingreso_log'] = np.log1p(
    df['ingreso'])
```

Rol 4: Class Balancer

 Balanceo de clases (PB-09)

Tareas del Sprint 2

1. Verificar el **ratio de clases** del target en el dataset limpio.
2. Decidir si se necesita balanceo (si la minoritaria $< 30\%$).
3. Aplicar **una o varias** técnicas y comparar:
 - **Oversampling**: RandomOverSampler, SMOTE, ADASYN.
 - **Undersampling**: RandomUnderSampler, NearMiss.
 - **Híbrido**: SMOTEENN, SMOTETomek.
4. **Importante**: el balanceo se aplica **solo al train set**, nunca al test.
5. Guardar dataset balanceado en data/processed/.

Ejemplo con SMOTE

```
from imblearn.over_sampling \
    import SMOTE
from sklearn.model_selection \
    import train_test_split

X_train, X_test, y_train, y_test = (
    train_test_split(
        X, y, test_size=0.2,
        stratify=y,
        random_state=42))

smote = SMOTE(random_state=42)
X_train_bal, y_train_bal = (
    smote.fit_resample(
        X_train, y_train))

print(y_train_bal.value_counts())
```

Rol 4: Class Balancer — Cuidado con Data Leakage

 Por qué el orden de las operaciones importa

Cuidado: Data Leakage

Data leakage: jamás balancear antes del *train/test split*. Si se aplica SMOTE (u otra técnica de oversampling) sobre el dataset completo **antes** de separar train y test, las muestras sintéticas generadas a partir de instancias del test “contaminan” el conjunto de entrenamiento, inflando artificialmente las métricas de evaluación y creando un modelo que parece bueno en el laboratorio pero falla en producción.

Regla

El orden correcto es siempre:

1. Hacer el `train_test_split` con `stratify=y`.
2. Aplicar el balanceo (SMOTE, undersampling, etc.) **solo** sobre `X_train`, `y_train`.
3. Dejar `X_test`, `y_test` con su distribución original (realista).
4. Evaluar el modelo sobre el test *desbalanceado* usando métricas robustas (F1, AUC-PR, recall por clase).

Rol 5: Pipeline Builder

 Pipeline reproducible con scikit-learn

Tareas del Sprint 2

1. Integrar las transformaciones de los roles 2, 3 y 4 en un `sklearn.Pipeline`.
2. Usar `ColumnTransformer` para aplicar distintas transformaciones a columnas numéricas y categóricas.
3. Encapsular lógica reutilizable en `src/preprocessing.py`.
4. Validar que el pipeline se entrena con `fit` y se aplica con `transform` sin errores.
5. Persistir el pipeline con `joblib.dump` para los Sprints 3 y 4.
6. Versionar el dataset procesado con **DVC**.

Pipeline ejemplo

```
from sklearn.pipeline import Pipeline
from sklearn.compose import (
    ColumnTransformer)
from sklearn.preprocessing import (
    StandardScaler, OneHotEncoder)
from sklearn.impute import SimpleImputer

num_pipe = Pipeline([
    ('imp', SimpleImputer(
        strategy='median')),
    ('sc', StandardScaler()),
])
cat_pipe = Pipeline([
    ('imp', SimpleImputer(
        strategy='most_frequent')),
    ('oh', OneHotEncoder(
        handle_unknown='ignore')),
])
preproc = ColumnTransformer([
    ('num', num_pipe, num_cols),
    ('cat', cat_pipe, cat_cols),
])
preproc.fit(X_train)
```

Tablero Kanban en GitHub Projects

Visualización del flujo de trabajo del Sprint 2



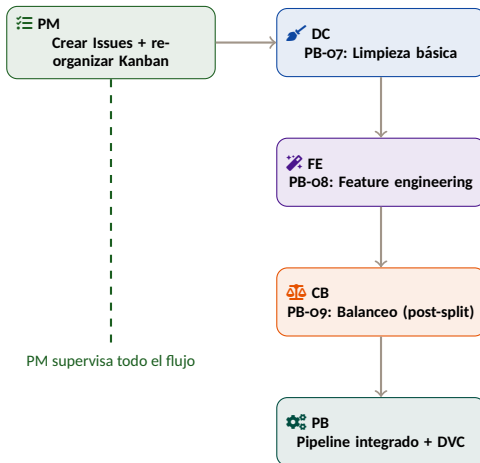
Flujo de Trabajo en el Kanban

Flujo

Cada Issue se mueve: **To Do** → se crea rama `feat/PB-XX` → **In Progress** → se abre Pull Request → **In Review** → se aprueba y mergea → **Done**. El Pipeline Builder integra al final, una vez los PRs anteriores están mergeados.

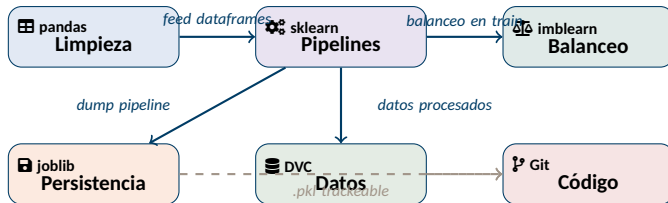
Dependencias entre Tareas

Orden sugerido de ejecución en el Sprint 2



Herramientas del Sprint 2: ¿Cómo encajan juntas?

Visión integrada para Data Preparation



💡 Idea clave

El Pipeline de scikit-learn es el **contrato** entre los datos crudos y el modelo. Si el pipeline está bien hecho, los Sprints 3 y 4 serán mucho más fáciles.

Tip 1: Train/Test split antes de transformar

✂ El error #1 de los principiantes en preparación de datos

✘ Mal: data leakage

```
# MAL: escalar antes de split
from sklearn.preprocessing \
    import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_tr, X_te, y_tr, y_te = (
    train_test_split(
        X_scaled, y))
# El test "vio" la media
# y std del train -> leakage
```

✔ Bien: split primero

```
# BIEN: split antes de fit
X_tr, X_te, y_tr, y_te = (
    train_test_split(
        X, y, stratify=y))

scaler = StandardScaler()
X_tr_s = scaler.fit_transform(X_tr)
# fit SOLO en train
X_te_s = scaler.transform(X_te)
# transform en test
```

⚠ Regla de oro

Toda transformación que aprenda parámetros del dato (fit) debe entrenarse **solo** sobre el conjunto de entrenamiento. El Pipeline de sklearn lo hace automáticamente.

Tip 2: ColumnTransformer + Pipeline

⚙️ Tratar numéricos y categóricos en un solo objeto

☰ Por qué

- Cada tipo de variable necesita un trato distinto.
- ColumnTransformer aplica un sub-pipeline a cada grupo de columnas.
- Combinado con Pipeline se obtiene un **único objeto** entrenable y serializable.
- Reutilizable en Sprints 3, 4 y 6 (deployment).

⚡ Persistir y reusar

```
import joblib

# Guardar
joblib.dump(preproc,
            'models/preproc.pkl')

# Cargar en otro notebook
preproc = joblib.load(
    'models/preproc.pkl')
X_new_t = preproc.transform(X_new)
```



Tip

Versiona el .pkl con DVC, no con Git, si es grande.

Tip 3: DVC para datasets procesados

 Versionar cada etapa del dataset

Etapas a versionar

- data/raw/ (del Sprint 1)
- data/interim/ (post-limpieza)
- data/processed/ (listo para modelar)

Cada etapa permite **auditar** qué cambió entre versiones del dataset.

>_ Comandos DVC

```
# Tras generar nuevo dataset
dvc add data/processed/train.csv
dvc add data/processed/test.csv
git add data/processed/*.dvc
git commit -m "data: dataset
    procesado Sprint 2"
git push && dvc push
```

Tip 4: De notebook a módulo Python

 Por qué migrar a `src/preprocessing.py`

Notebook (exploración)

Los notebooks son útiles para:

- Iterar y experimentar.
- Visualizar resultados.
- Documentar decisiones.

Pero **no** son ideales para:

- Reutilizar código.
- Hacer tests.
- Importar funciones desde otro notebook.

Módulo Python (producción)

Migrar a `src/preprocessing.py`:

- Funciones `build_preprocessor()`, `clean_data()`, etc.
- Importar desde el notebook con `from src.preprocessing import ...`
- Reutilizable en Sprints 3, 4, 6.
- Testeable con `pytest`.



Tip

Regla práctica: si una función aparece en más de un notebook, **mígrala** a `src/`.

Tip 5: Sincronización de dependencias — Caso práctico

 Cuando un compañero agrega imbalanced-learn al environment.yml

Escenario: Ana (*Class Balancer*) necesita imbalanced-learn para aplicar SMOTE en PB-09. El paquete aún no está en el entorno del equipo.

Ana: instala, exporta y hace push

```
# 1. Crear rama para la tarea
git checkout -b feat/PB-09

# 2. Activar el entorno del proyecto
conda activate proyecto

# 3. Instalar el paquete nuevo
conda install -c conda-forge \
    imbalanced-learn

# 4. Re-exportar environment.yml
conda env export --from-history \
    > environment.yml

# 5. Verificar que aparece en el YML
grep imbalanced environment.yml

# 6. Commit y push
git add environment.yml
git commit -m "chore(PB-09): agrega
    imbalanced-learn al entorno"
git push origin feat/PB-09

# 7. Abrir Pull Request en GitHub
```

Resto del equipo: pull + update

```
# Tras mergear el PR a main:

# 1. Sincronizar el repo local
git checkout main
git pull origin main

# 2. Activar el entorno
conda activate proyecto

# 3. Actualizar el entorno
# desde el YML modificado
conda env update -f environment.yml \
    --prune

# 4. Verificar la instalacion
python -c "import imblearn; \
    print(imblearn.__version__)"

# 5. Reanudar el trabajo en su rama
git checkout feat/PB-08
git merge main # traer cambios
```

Errores comunes en Data Preparation

! Qué evitar y cómo resolverlo

	Error común	Solución
⊘	Imputar / escalar antes del <code>train_test_split</code>	Hacer split primero, luego <code>fit</code> solo en train
⊘	Aplicar SMOTE al test set	SMOTE solo sobre el train; el test debe ser realista
⊘	Eliminar nulos sin entender por qué existen	Investigar el mecanismo (MCAR/MAR/MNAR) e imputar con criterio
⊘	Hacer feature engineering sin documentar	Cada feature debe tener una hipótesis de por qué ayudaría al modelo
⊘	Sobrescribir el dataset original	Trabajar siempre sobre una copia ; guardar etapas en <code>interim/</code>
⊘	Duplicar código entre notebooks	Migrar funciones reutilizables a <code>src/preprocessing.py</code>

Entregables del Sprint 2

Lo que cada rol debe entregar al final de la semana

Rol	Entregable	Formato
PM	Issues del Sprint 2 cerrados, board Kanban actualizado, Sprint Review preparado	GitHub Project
DC	Notebook de limpieza con justificación de cada decisión, dataset intermedio versionado	05_data_cleaning.ipynb
FE	Notebook con features nuevas documentadas (nombre, cómo se calcula, hipótesis)	06_feature_eng.ipynb
CB	Notebook con análisis de balance, técnica aplicada y comparación antes/después	07_class_balance.ipynb
PB	Pipeline integrado en src/preprocessing.py, notebook de validación y .pkl persistido	08_pipeline.ipynb + src/

Definition of Done (DoD)

Criterios que cada tarea debe cumplir para considerarse terminada

✓ Criterios Generales

- ☐ Código en rama `feat/PB-XX` con Pull Request aprobado y mergeado a `main`.
- ☐ Notebook ejecutable de principio a fin sin errores (*Restart & Run All*).
- ☐ Celdas markdown documentando **cada decisión** de transformación.
- ☐ El entorno se reproduce con `conda env create -f environment.yml`.
- ☐ Issue correspondiente movido a "Done" en el board.

★ Criterios específicos del Sprint 2

- ☐ Train/test split realizado **antes** de cualquier `fit`.
- ☐ Pipeline final entrenable con `fit` y aplicable con `transform`.
- ☐ Dataset procesado guardado en `data/processed/` y versionado con DVC.
- ☐ Pipeline persistido con `joblib` y reproducido en notebook independiente.
- ☐ Cada feature nueva documentada con su **hipótesis** de utilidad.

Visión General: 6 Sprints hasta el Despliegue

Del entendimiento del negocio al MVP desplegado en AWS

Sprint	Fase CRISP-DM	Objetivo principal
1	Business + Data Underst.	Comprender el problema y explorar los datos
2	Data Preparation	Limpiar, transformar y crear features
3	Modeling	Entrenar modelos baseline y comparar
4	Modeling	Optimizar hiperparámetros y modelos avanzados
5	Evaluation	Evaluar Business Value y documentar resultados
6	Deployment	Desplegar el mejor modelo como MVP en AWS

Meta del Sprint 2

Entregar un **pipeline reproducible** y un **dataset listo para modelar**, de modo que el Sprint 3 pueda enfocarse exclusivamente en entrenar y comparar algoritmos sin tener que reescribir transformaciones.

Resumen del Sprint 2

❓ ¿Qué?

Data Preparation: limpieza, features, balanceo y pipeline

👥 ¿Quién?

5 roles: PM, DC, FE, CB, PB

⚙️ ¿Cómo?

scikit-learn Pipelines + DVC + GitHub



Entregable

4 Notebooks +
src/preprocessing.py
+ dataset versionado



Siguiente

Sprint 3: Modeling (baselines)

! Recordar

- El Sprint 2 **aún no** entrena modelos. Se trata de **preparar** antes de modelar.
- Toda transformación que aprende del dato debe entrenarse **solo** sobre el train set.
- Un pipeline bien diseñado en este sprint ahorrará horas en los Sprints 3, 4 y 6.

Checklist de Inicio

Pasos inmediatos para comenzar el Sprint 2

1. **PM:** Cerrar el Sprint 1 en el board y crear los Issues del Sprint 2 (PB-07, PB-08, PB-09 y sub-issues).
2. **PM:** Reasignar roles para que cada miembro experimente una nueva fase.
3. **Todos:** `git pull && dvc pull` para sincronizar el estado del Sprint 1.
4. **Todos:** Actualizar el entorno con `conda env update -f environment.yml` (agregar `imbalanced-learn`, `feature-engine`).
5. **DC:** Iniciar `05_data_cleaning.ipynb` en rama `feat/PB-07` a partir de los hallazgos del EDA del Sprint 1.
6. **FE:** Una vez mergeado PB-07, iniciar `06_feature_eng.ipynb` en rama `feat/PB-08`.
7. **CB:** Tras feature engineering, ejecutar el train/test split y aplicar balanceo en `07_class_balance.ipynb` (rama `feat/PB-09`).
8. **PB:** Integrar todo en `src/preprocessing.py` y validar en `08_pipeline.ipynb` (rama `feat/PIPE`).
9. **Todos:** Sprint Review — demostrar el pipeline corriendo end-to-end sobre datos crudos.



Diplomado de Especialización en **Data Analytics**